

DATABASE SYSTEM PROJECT - IAMMARINO ANTONIO

Requirements Collection	3
Requirements Analysis	3
Choice of the Right Level of Abstraction	3
Output (after choosing the correct level of abstraction)	4
Standardize sentence structure	5
Output (after sentence standardization)	6
Linearize sentences, split complex ones and explicit references	7
Identify homonyms and synonyms and unify terms	8
Output (Final requirements after linguistic analysis)	8
Glossary of Terms	9
Phrases grouped by Concept	9
Conceptual Design	11
Skeleton Schema	11
Final schema	11
Business Rules and Constraints	12
Logical Design	13
Volumes Table	13
Table of Accesses	14
Op1	14
Op2	15
Op3	15
Op4	15
Redundancy Analysis	15
OP1 with capacity check without redundancy	16
OP1 with capacity check with redundancy	16
Schema Restructuring	16
UML Schema	18
Implementation	18
Composite Types	18
Collections: VARRAY and Nested Tables	18
Business Rules Implementation	21
Declarative Constraints	21
Triggers and Procedures	22
Barn Capacity Limit (BR4)	23
Chronological Order (BR5)	25
log_tool_usage Procedure	29
Deal Limitations and Space Vacation (BR6-7-8-9)	30
register_animal_deal Procedure	33
Views	35
Barn Tools Inventory (Operation 2)	35
Employee Tool Usage (Operation 4)	35
Physical Design	36
Operation 2: Tool in a specific Barn	39

Operation 4: Tool usage for a specific employee	39
Index Creation	40
Post-Index Execution Plans	41
Operation 2	41
Operation 4	42
Physical Storage Analysis	42
Data Warehouse	44
Constellation Diagram	44
Web Application	45
Architecture	45
Application Sections	46
Dashboard	46
Animals	47
Barn Tools	48
Tool Usage	48

Requirements Collection

The Ranch Holdings farm specializes in animal breeding. For each animal, the following information is known: the identification code, the weight expressed in kilograms, the sex, and the date of birth. In particular, animals are classified into swine, cattle, and horses. For horses, the date of the last shoeing must be tracked in order to schedule the next one.

Each animal lives in a barn. For each barn, the geographic location and the surface area occupied, expressed in square meters, are known. Each barn is equipped with the tools necessary for its maintenance. In particular, for each tool, the description, the type of tool, its purchase price, and the barn where it is located must be tracked. Each tool is identified by a code.

Several employees work at the farm. For each employee, personal data is known, including the tax code, first name, last name, address (composed of street, postal code, city, and province), and phone number, as well as the monthly salary received. Employees are divided into two categories: maintenance staff and those who take care of the animals. Maintenance staff use the tools located in the various barns to carry out their tasks; for each use of a tool, the date of use must be tracked. For the employees who take care of the animals, the list of animals they take care of must be maintained. Note that each employee may, at different times of the day, be involved in both maintenance and animal care, depending on the needs of the moment.

The Ranch Holdings farm maintains business relationships with external companies for the purchase and sale of animals. Each of these companies is characterized by a VAT number, a name, an address (composed of street, postal code, city, and province), and one or more phone numbers. Whenever an animal is purchased, the purchase date and the price paid must be recorded; similarly, in case of sale, the date and the selling price are tracked.

Requirements Analysis

Choice of the Right Level of Abstraction

- Regarding the geographic location of the barns, the phrase is broad. Therefore, I assume that it represents a descriptive string indicating the specific operational area or sector within the farm estate, rather than precise topographical coordinates.

- Regarding the type of tool, since there is not a specific list of characteristics, I assume it represents a general category of usage. Examples of tool types can be "*cleaning equipment*", "*feeding machinery*", or "*veterinary devices*".
- Regarding the external companies' phone numbers, the specification "*one or more phone numbers*" is general. Therefore, I assume that a company can simply have a list of contact numbers directly associated with its profile.

Output (after choosing the correct level of abstraction)

The Ranch Holdings farm specializes in animal breeding. For each animal, the following information is known: the identification code, the weight expressed in kilograms, the sex and the date of birth. In particular, animals are classified into swine, cattle, and horses. For horses, the date of the last shoeing must be tracked in order to schedule the next one.

Each animal lives in a barn. For each barn, the geographic location **intended as a descriptive operational area** within the farm estate and the surface area occupied, expressed in square meters, are known. Each barn is equipped with the tools necessary for its maintenance. In particular, for each tool, the description, the type of tool **representing a general category**, its purchase price, and the barn where it is located must be tracked. Each tool is identified by a code.

Several employees work at the farm. For each employee, personal data is known, including the tax code, first name, last name, address (composed of street, postal code, city, and province), and phone number, as well as the monthly salary received. Employees are divided into two categories: maintenance staff and those who take care of the animals. Maintenance staff use the tools located in the various barns to carry out their tasks; for each use of a tool, the date of use must be tracked. For the employees who take care of the animals, the list of animals they take care of must be maintained. Note that each employee may, at different times of the day, be involved in both maintenance and animal care, depending on the needs of the moment.

The Ranch Holdings farm maintains business relationships with external companies for the purchase and sale of animals. Each of these companies is characterized by a VAT number, a name, an address (composed of street, postal code, city, and province), and **a list of contact phone numbers**. Whenever an animal is purchased, the purchase date and the price paid must be recorded; similarly, in case of sale, the date and the selling price are tracked.

Standardize sentence structure

Pattern used for standardization: *For [concept], we handle [properties]*

Main sentence	Sentence transformed
The Ranch Holdings farm specializes in animal breeding. For each animal, the following information is known: the identification code, the weight expressed in kilograms, the sex and the date of birth.	The Ranch Holdings farm specializes in animal breeding. For animals, we handle the identification code, weight expressed in kilograms, sex and date of birth.
In particular, animals are classified into swine, cattle, and horses.	Animals can be classified as swine, cattle, or horses.
For horses, the date of the last shoeing must be tracked in order to schedule the next one.	For horses, we handle the date of the last shoeing.
Each animal lives in a barn. For each barn, the geographic location intended as a descriptive operational area within the farm estate and the surface area occupied, expressed in square meters, are known.	Each animal lives in a barn. For barns, we handle the geographic location as an operational area, and surface area expressed in square meters.
Each barn is equipped with the tools necessary for its maintenance. In particular, for each tool, the description, the type of tool representing a general category like cleaning equipment or feeding machinery, its purchase price, and the barn where it is located must be tracked. Each tool is identified by a code.	Each barn is equipped with tools. For tools, we handle the unique code, description, type of tool as a general category, purchase price, and the specific barn where it is located.
Several employees work at the farm. For each employee, personal data is known, including the tax code, first name, last name, address (composed of street, postal code, city, and province), and phone number, as well as the monthly salary received.	Employees work at the farm. For employees, we handle the tax code, first name, last name, street, postal code, city, province, phone number, and monthly salary.
Employees are divided into two categories: maintenance staff and those who take care of the animals.	Employees can be classified as maintenance staff or those who take care of the animals.

Maintenance staff use the tools located in the various barns to carry out their tasks; for each use of a tool, the date of use must be tracked.	Maintenance staff use tools. For tool usage, we handle the date of use.
For the employees who take care of the animals, the list of animals they take care of must be maintained.	For those who take care of the animals, we handle the list of animals they take care of.
Note that each employee may, at different times of the day, be involved in both maintenance and animal care, depending on the needs of the moment.	Employees can flexibly perform both maintenance and animal care roles.
The Ranch Holdings farm maintains business relationships with external companies for the purchase and sale of animals.	The farm maintains business relationships with external companies for the purchase and sale of animals.
Each of these companies is characterized by a VAT number, a name, an address (composed of street, postal code, city, and province), and a list of contact phone numbers.	For an external company, we handle the VAT number, name, street, postal code, city, province, and list of contact phone numbers.
Whenever an animal is purchased, the purchase date and the price paid must be recorded; similarly, in case of sale, the date and the selling price are tracked.	For an animal purchase, we handle the purchase date and price paid. For an animal sale, we handle the date and selling price.

Output (after sentence standardization)

The Ranch Holdings farm specializes in animal breeding. For animals, we handle the identification code, weight expressed in kilograms, sex and date of birth. Animals can be classified as swine, cattle, or horses. For horses, we handle the date of the last shoeing.

Each animal lives in a barn. For barns, we handle the geographic location as an operational area, and surface area expressed in square meters. Each barn is equipped with tools. For tools, we handle the unique code, description, type of tool as a general category, purchase price, and the specific barn where it is located.

Employees work at the farm. For employees, we handle the tax code, first name, last name, street, postal code, city, province, phone number, and monthly salary. Employees can be classified as maintenance staff or those who take care of the animals. Maintenance staff use tools. For tool usage, we handle the date of use. For those who take care of the animals, we handle the list of animals they take care of.

Employees can flexibly perform both maintenance and animal care roles.

The farm maintains business relationships with external companies for the purchase and sale of animals. For an external company, we handle the VAT number, name, street, postal code, city, province, and list of contact phone numbers. For an animal purchase, we handle the purchase date and price paid. For an animal sale, we handle the date and selling price.

Linearize sentences, split complex ones and explicit references

Original Sentence	Linearized Sentence
For tools, we handle the unique code, description, type of tool as a general category, purchase price, and the specific barn where it is located.	For tools, we handle the unique code, description, type, and purchase price. Each tool is located in a specific barn.
For those who take care of the animals, we handle the list of animals they take care of.	For animal care staff members, we handle the list of animals they take care of.
The farm maintains business relationships with external companies for the purchase and sale of animals.	The farm purchases animals from external companies. The farm sells animals to external companies.
For employees, we handle the tax code, first name, last name, street, postal code, city, province, phone number, and monthly salary.	For employees, we handle the tax code, first name, last name, employee street, employee postal code, employee city, employee province, phone number, and monthly salary.
For an external company, we handle the VAT number, name, street, postal code, city, province, and list of contact phone numbers.	For an external company, we handle the VAT number, company name, company street, company postal code, company city, company province, and list of contact phone numbers.

For an animal purchase, we handle the purchase date and price paid.	For an animal purchase, we handle the purchase date and purchase price.
For an animal sale, we handle the date and selling price.	For an animal sale, we handle the sale date and sale price.

Identify homonyms and synonyms and unify terms

Original Terms	Unified Term
Employee, staff members, member	Employee

Output (Final requirements after linguistic analysis)

The Ranch Holdings farm specializes in animal breeding. For animals, we handle the identification code, weight expressed in kilograms, sex, and date of birth. Animals can be classified as swine, cattle, or horses. For horses, we handle the date of the last shoeing.

Each animal lives in a barn. For barns, we handle the geographic location as an operational area, and surface area expressed in square meters. Each barn is equipped with tools. For tools, we handle the unique code, description, type, and purchase price. Each tool is located in a specific barn.

Employees work at the farm. For employees, we handle the tax code, first name, last name, employee street, employee postal code, employee city, employee province, phone number, and monthly salary. Employees can be classified as maintenance employees or animal care employees. Maintenance employees use tools. For tool usage, we handle the usage date. For animal care employees, we handle the list of animals they take care of. Employees can flexibly perform both maintenance and animal care roles.

The farm purchases animals from external companies. The farm sells animals to external companies. For an external company, we handle the VAT number, company name, company street, company postal code, company city, company province, and list of contact phone numbers. For an animal purchase, we handle the purchase date and purchase price. For an animal sale, we handle the sale date and sale price.

Glossary of Terms

Concept	Description	Connections
Animal	A living creature bred on the farm. Can be swine, cattle or horse.	Barn, Animal care employee, External Company
Barn	A physical area where animals live and tools are kept.	Animal, Tool
Tool	Equipment used for farm maintenance tasks.	Barn, Maintenance employee
Employee	A person who works at the farm.	Maintenance employee, Animal care employee
Maintenance employee	An employee role focused on using tools for tasks.	Tool, Employee
Animal care employee	An employee role focused on taking care of animals.	Animal, Employee
External company	A business partner that sells or purchases animals from our farm.	Animal

Phrases grouped by Concept

Phrases related to Animal
The Ranch Holdings farm specializes in animal breeding. For animals, we handle the identification code, weight expressed in kilograms, sex, and date of birth. Animals can be classified as swine, cattle, or horses. For horses, we handle the date of the last shoeing. Each animal lives in a barn. For animal care employees, we handle the list of animals they take care of. The farm purchases animals from external companies. The farm sells animals to external companies. For an animal purchase, we handle the purchase date and purchase price. For an animal sale, we handle the sale date and sale price.

Phrases related to Barn

Each animal lives in a barn. For barns, we handle the geographic location as an operational area, and surface area expressed in square meters. Each barn is equipped with tools.

Phrases related to Tool

Each barn is equipped with tools. For tools, we handle the unique code, description, type, and purchase price. Each tool is located in a specific barn.

Maintenance employees use tools. For tool usage, we handle the usage date.

Phrases related to Employee

Employees work at the farm. For employees, we handle the tax code, first name, last name, employee street, employee postal code, employee city, employee province, phone number, and monthly salary. Employees can be classified as maintenance employees or animal care employees. Maintenance employees use tools. For tool usage, we handle the usage date. For animal care employees, we handle the list of animals they take care of. Employees can flexibly perform both maintenance and animal care roles.

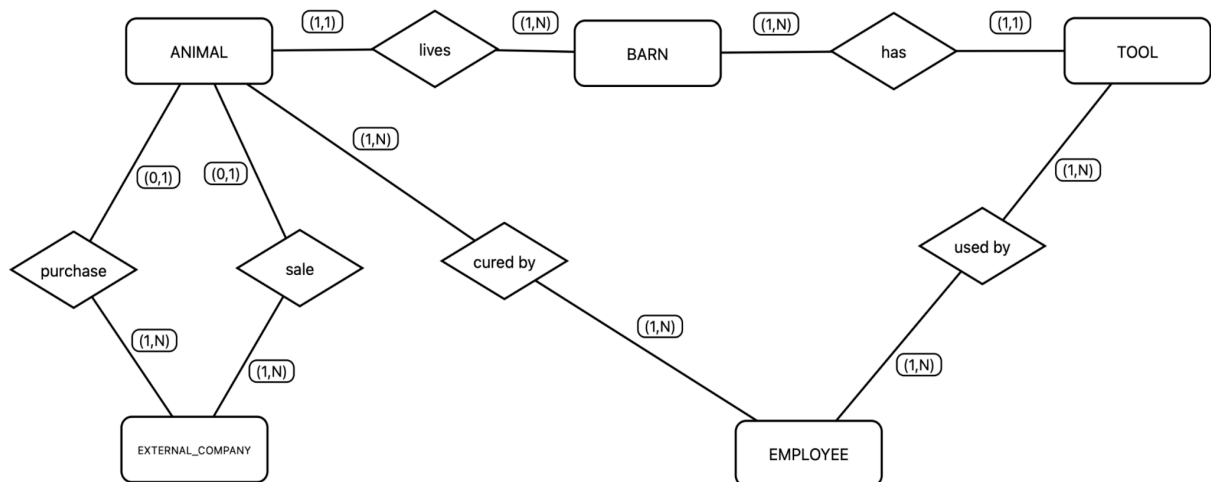
Phrases related to External Company

The farm purchases animals from external companies. The farm sells animals to external companies. For an external company, we handle the VAT number, company name, company street, company postal code, company city, company province, and list of contact phone numbers. For an animal purchase, we handle the purchase date and purchase price. For an animal sale, we handle the sale date and sale price.

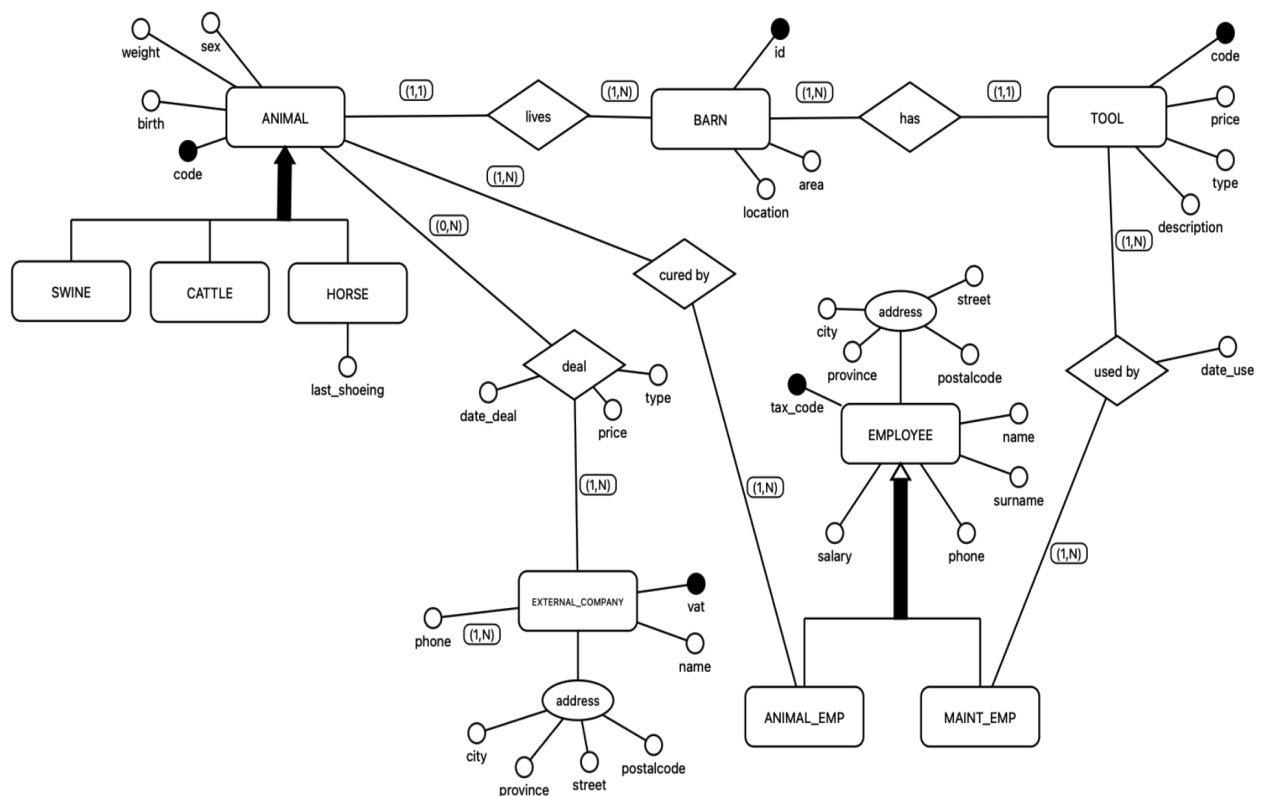
Conceptual Design

For the conceptual design, I followed a hybrid strategy. First, I used a top-down approach to identify the main components and represent them in a skeleton schema. Then, I refined each separate component with a bottom-up approach, and finally, I integrated them following the skeleton.

Skeleton Schema



Final schema



During the integration phase of the local schemas into the final conceptual model, the initially distinct *purchase* and *sale* relationships were unified into a single relationship named **deal**. To distinguish the exact nature of the transaction, a **type** attribute was introduced, which accepts specific values such as **S** (Sale) or **P** (Purchase). The cardinality on the ANIMAL side is defined as (0,N). The minimum cardinality is 0, as an animal might be born directly on the farm (meaning it is never purchased) and might remain there (meaning it is never sold). The maximum cardinality is N, because an animal could potentially be purchased from one company and later sold to another, thus participating in multiple distinct occurrences of the deal relationship.

Generalizations:

- Animal Specialization: The generalization for the ANIMAL entity (into SWINE, CATTLE, and HORSE) is modeled as **total and exclusive** (disjoint).
- Employee Specialization: The generalization for the EMPLOYEE entity (into MAINT_EMP and ANIMAL_EMP) is modeled as **total and inclusive** (overlapping). This directly satisfies the requirement stating that employees can flexibly perform both maintenance and animal care roles depending on the farm's needs.

Attribute Constraints: The domain of the sex attribute within the ANIMAL entity has been restricted to specific predefined values, specifically *M* (Male) or *F* (Female).

Business Rules and Constraints

(BR1) The type of a deal must be restricted to the predefined values: *S* or *P*.

(BR2) The sex attribute of an ANIMAL must be restricted to specific predefined values: *M* or *F*.

(BR3) All financial attributes (price in deal, price in TOOL, salary in EMPLOYEE) and physical measurement attributes (weight in ANIMAL, area in BARN) must be greater than zero.

(BR4) The number of animals currently living in a specific BARN must not exceed a maximum capacity threshold, which is calculated based on the barn's area (a maximum of 1 animal per 4 square meters).

(BR5) The dates recorded in the database must follow logical chronological order. For instance, an animal's *date_deal* cannot be earlier than its birth date. Similarly, for a HORSE, the *last_shoeing* date cannot be in the future. The *date_use* recorded when a Maintenance Employee uses a TOOL must be less than or equal to the current system date. It is possible to log present or past usage, but never future usage.

(BR6) An ANIMAL can participate in a *Sale* deal a maximum of one time in its lifecycle. Once an animal is sold, it cannot be sold again by the farm unless it is repurchased as a new distinct entity.

(BR7) An ANIMAL can participate in a *Purchase* deal a maximum of one time in its lifecycle. Once an animal is purchased, it cannot be purchased again by the farm unless it is sold and repurchased as a new distinct entity.

(BR8) Once a deal is finalized, its core attributes (such as price and date) cannot be modified in the future.

(BR9) An ANIMAL that has been sold to an EXTERNAL_COMPANY must logically vacate its space in the barn. Therefore, its association in the *lives* relationship must be archived or terminated to free up capacity for BR4.

Logical Design

Most frequent operations:

Operation 1: Insert a new animal into the farm, including its species (swine, cattle, or horse), weight, sex, date of birth, and the barn where it will be housed - *15 times per day*.

Operation 2: For each barn, print the list of tools available, including description, tool type, and purchase price - *30 times per day*.

Operation 3: Insert a new tool usage record by a maintenance employee, including the date of use - *60 times per day*.

Operation 4: For each maintenance employee, print the list of all tools they have used, including the tool's description, type, the barn where it is located, and the date of each use - *30 times per day*.

Each barn houses an average of 25 animals, is equipped with an average of 10 tools, the farm employs an average of 50 employees (40% of whom are maintenance staff, with the rest taking care of the animals), each tool is used an average of 3 times per day, 10% of the animals are horses, the farm maintains relationships with an average of 200 external companies, and each company has carried out an average of 10 transactions (purchases/sales) in the period considered.

Volumes Table

Concept	Type	Volume	Assumption
External_Company	E	200	/
Deal	R	2000	/
Animal	E	1500	1,500 animals were purchased, and

			500 of them were also sold. No animals were born naturally on the farm during the first 30 days of activity
Lives	R	1500	/
Barn	E	60	/
Has	R	600	/
Tool	E	600	/
Employee	E	50	/
Maint_Emp	E	20	/
Animal_Emp	E	30	/
Used_by	R	54000	600*3*30 days. I assume 30 days and each daily use of a tool is performed by a different employee.
Cured_by	R	3000	On average, two employees take care of a single animal.
Horse	E	150	/
Swine	E	700	/
Cattle	E	650	/

Table of Accesses

Operation	Type	Frequency	Assumption
1	i	15 times/day	/
2	b	30 times/day	This reporting analysis can be scheduled periodically (e.g., 3 times an hour).
3	i	60 times/day	/
4	b	30 times/day	Also this analysis can be scheduled.

Op1

Concept	Type	Accesses	Type Accesses
Barn	E	1	R

Lives	R	1	W
Animal	E	1	W

Op2

Concept	Type	Accesses	Type Accesses
Barn	E	60	R
Has	R	600	R
Tool	E	600	R

Op3

Concept	Type	Accesses	Type Accesses
Tool	E	1	R
Maint_Emp	E	1	R
Used_by	R	1	W

Op4

Concept	Type	Accesses	Type Accesses
Maint_Emp	E	20	R
Used_by	R	54.000	R
Tool	E	54.000	R
Has	R	54.000	R
Barn	E	54.000	R

Redundancy Analysis

Based strictly on the four explicitly requested data operations, no natural redundancies are present in the conceptual schema. However, a deeper analysis of the system's **Business Rules** (specifically (BR4) regarding the maximum barn capacity and (BR9) regarding space vacation after a sale) reveals that the system will frequently need to compute the **current number of animals in a specific barn**.

Every time Operation 1 is executed (*Insert a new animal, 15 times/day*), the system must verify the capacity constraint before allowing the insertion. If evaluated *without* redundancy, verifying this constraint would require reading all the existing occurrences in the **lives** relationship for that specific barn (an average of 25 reads per run) just to perform a COUNT.

OP1 with capacity check without redundancy

Concept	Type	Accesses	Type Accesses
Barn	E	1	R
Lives	R	25	R
Lives	R	1	W
Animal	E	1	W

OP1 with capacity check with redundancy

By introducing the redundant attribute **current_occupancy** within the BARN entity, the capacity check becomes an immediate read of the attribute, followed by an additional write operation to increment the counter upon successful insertion.

Concept	Type	Accesses	Type Accesses
Barn	E	1	R
Animal	E	1	W
Lives	R	1	W
Barn	E	1	W

Given that write operations are generally more expensive than read operations, but the capacity check acts as a strict blocking constraint for every daily insertion, maintaining this redundant attribute drastically reduces the computational overhead of the COUNT aggregations, ensuring system efficiency.

Schema Restructuring

Regarding the ANIMAL generalization hierarchy, since only the *Horse* possesses a specific attribute (*last_shoeing*) a flat structure is more efficient. Consequently, the ANIMAL entity has been enriched with a **type** attribute, which takes the values *Swine*, *Cattle*, or *Horse*, and the **last_shoeing** attribute, which will remain set to *NULL* for all instances not belonging to the equine species.

I have also chosen to eliminate the sub-entities MAINT_EMP and ANIMAL_EMP in favor of a single, unified EMPLOYEE entity. The requirements explicitly state that an employee may alternate between or simultaneously perform maintenance and animal care tasks depending on the time of day. This implies an overlapping and highly dynamic specialization.

Oracle does not natively support overlapping inheritance, meaning a single object instance cannot simultaneously belong to multiple distinct subtypes (an employee cannot be both a *maintenance_employee_t* and an *animal_employee_t* at the database type level).

Implementing separate tables for each role would require duplicating the employee's personal and salary data, inevitably leading to **update anomalies** (e.g., updating a salary in one table but forgetting the other).

Alternatively, using **boolean flags** (e.g., *is_maintenance*) within a single table was discarded. If an employee registered in the *used_by* table later has their *is_maintenance* flag set to *false*, the database would suffer from a semantic integrity breach, as the tuple in the usage table would lose its logical justification.

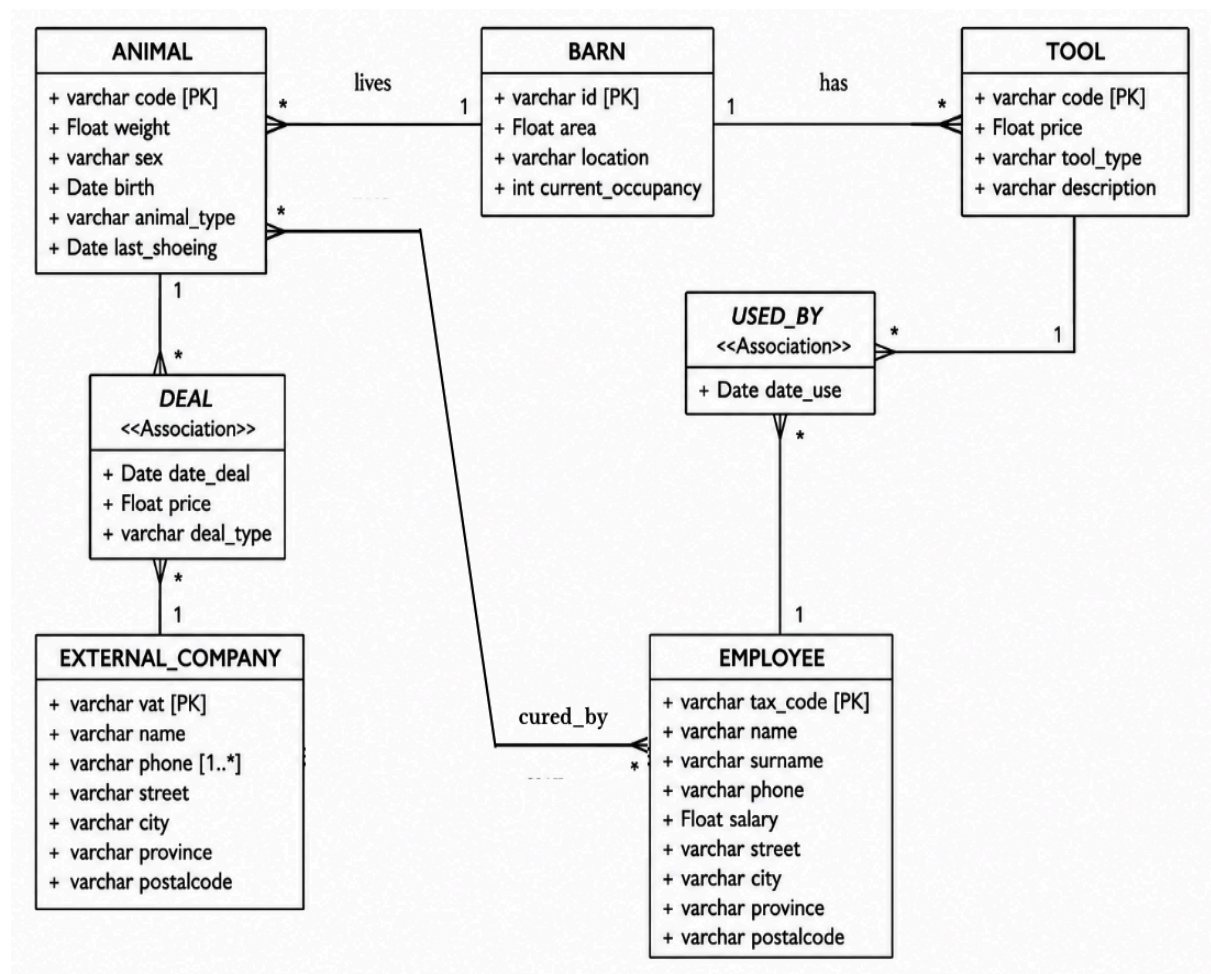
Furthermore, since the conceptual sub-entities lacked specific attributes, adding structural complexity (flags or subtypes) was unjustified.

To resolve these issues, the employee's role is no longer modeled as a static structural property. Instead, it emerges implicitly from their participation in business relationships. The *used_by* (for tool maintenance) and *cured_by* (for animal care) associations are now linked directly to the parent EMPLOYEE entity.

This approach allows any staff member to act as a *maint_emp*, an *animal_emp* or both, while strictly preventing data duplication and preserving the semantic integrity of the database schema.

Finally, I have consolidated the **redundancy** identified during the performance analysis by officially adding the derived attribute *current_occupancy* to the BARN entity.

UML Schema



Implementation

Following the definition of the logical UML schema, the project advances to the implementation phase.

Composite Types

To manage address attributes (street, city, province, postal code), a type ***t_address*** was defined. This choice ensures code reusability: the same *t_address* object is instantiated within both the *t_employee* and *t_external_company* entities.

Collections: VARRAY and Nested Tables

To manage multivalued attributes and complex associations:

- **VARRAY (t_phone_varray)**: Used to store the phone numbers associated with an external company. Since a company possesses a limited and

predictable number of contact records (a maximum limit of 10 was assumed), the VARRAY represents the most efficient choice.

- **Nested Tables (*t_usage_nt*, *t_cured_nt*, *t_deal_nt*):** These were utilized to handle N:M relationships. Specifically:
 - The animals collection of type *t_cured_nt* was embedded within *t_employee*. The business requirements explicitly dictated that *"the list of animals they take care of must be maintained"*. By storing references (REF) to the cared-for animals directly within the employee record, this N:M relationship was resolved in a purely object-oriented manner.
 - Similarly, *t_usage_nt* is embedded within *t_employee* to resolve the N:M relationship with tools, encapsulating the *USED_BY* association and its specific *date_use* attribute.
 - Finally, *t_deal_nt* is embedded within *t_external_company* to encapsulate the *DEAL* relationship, maintaining the transaction history along with attributes such as price and date.

```
-- Address Type
CREATE OR REPLACE TYPE t_address AS OBJECT (
  street VARCHAR2(30),
  city VARCHAR2(30),
  province VARCHAR2(30),
  postalcode VARCHAR2(30)
);
/

-- External Company has a list of phones (assume max 10)
CREATE OR REPLACE TYPE t_phone_varray AS VARRAY(10) OF VARCHAR2(20);
/

CREATE OR REPLACE TYPE t_barn AS OBJECT (
  barn_id VARCHAR2(30),
  area NUMBER(8,2),
  barn_location VARCHAR2(30),
  current_occupancy INTEGER
);
/

CREATE OR REPLACE TYPE t_tool AS OBJECT (
  code VARCHAR2(30),
  price NUMBER(5,2),
  tool_type VARCHAR2(30),
  tool_description VARCHAR2(30),
  barn_ref REF t_barn
);
/

CREATE OR REPLACE TYPE t_usage AS OBJECT (
  tool_ref REF t_tool,
  date_use DATE
);
/
CREATE OR REPLACE TYPE t_usage_nt AS TABLE OF t_usage;
/
```

```

CREATE OR REPLACE TYPE t_animal AS OBJECT (
    code VARCHAR2(30),
    weight NUMBER(6,2),
    sex CHAR(1),
    birth DATE,
    animal_type VARCHAR2(30),
    last_shoeing DATE,
    barn_ref REF t_barn
);
/

CREATE OR REPLACE TYPE t_cured_nt AS TABLE OF REF t_animal;
/

CREATE OR REPLACE TYPE t_employee AS OBJECT (
    tax_code VARCHAR2(30),
    emp_name VARCHAR2(30),
    surname VARCHAR2(30),
    salary NUMBER(8,2),
    emp_address t_address,
    tools t_usage_nt,
    animals t_cured_nt
);
/

CREATE OR REPLACE TYPE t_deal AS OBJECT (
    animal_ref REF t_animal,
    date_deal DATE,
    price NUMBER(8,2),
    deal_type CHAR(1)
);
/

CREATE OR REPLACE TYPE t_deal_nt AS TABLE OF t_deal;
/

CREATE OR REPLACE TYPE t_external_company AS OBJECT (
    vat VARCHAR2(30),
    comp_name VARCHAR2(30),
    comp_address t_address,
    phones t_phone_varray,
    deals t_deal_nt
);
/

CREATE TABLE barns OF t_barn (
    CONSTRAINT barn_pk PRIMARY KEY (barn_id),
    CONSTRAINT barn_area_chk CHECK (area > 0)
);

CREATE TABLE tools OF t_tool (
    CONSTRAINT tool_pk PRIMARY KEY (code),
    CONSTRAINT tool_price_chk CHECK (price > 0),
    CONSTRAINT tool_barn_fk FOREIGN KEY (barn_ref) REFERENCES barns
);

CREATE TABLE employees OF t_employee (
    CONSTRAINT employee_pk PRIMARY KEY (tax_code),
    CONSTRAINT emp_salary_chk CHECK (salary > 0)
)
NESTED TABLE tools STORE AS emp_usages_nt_tab
NESTED TABLE animals STORE AS emp_cured_nt_tab;

```

```

CREATE TABLE animals OF t_animal (
  CONSTRAINT animal_pk PRIMARY KEY (code),
  CONSTRAINT animal_sex_chk CHECK (sex IN ('M', 'F')),
  CONSTRAINT animal_weight_chk CHECK (weight > 0),
  CONSTRAINT animal_type_chk CHECK (animal_type IN ('Swine', 'Cattle', 'Horse')),
  CONSTRAINT animal_barn_fk FOREIGN KEY (barn_ref) REFERENCES barns
);

CREATE TABLE external_companies OF t_external_company (
  CONSTRAINT ext_comp_pk PRIMARY KEY (vat)
)
NESTED TABLE deals STORE AS comp_deals_nt_tab;

```

```

ALTER TABLE comp_deals_nt_tab
ADD CONSTRAINT deal_type_chk CHECK (deal_type IN ('S', 'P'));

```

Business Rules Implementation

With the physical schema successfully instantiated, it is necessary to map the domain constraints to the database architecture.

Declarative Constraints

Static rules that rely exclusively on the data within a single row being inserted or updated were explicitly resolved during the DDL phase using **CHECK** constraints.

- **(BR1) Deal Type Restriction:** In Oracle's Object-Relational model, declarative constraints (such as CHECK) cannot be defined directly within a User-Defined Type (CREATE TYPE). Furthermore, constraints on attributes encapsulated within a collection cannot be declared inline during the creation of the parent object table (CREATE TABLE external_companies). Therefore, to strictly enforce the restriction of the *deal_type* attribute to either 'S' (Sale) or 'P' (Purchase), a two-step approach was required. First, the nested table was allocated to an out-of-line physical storage table (*comp_deals_nt_tab*) via the *STORE AS* clause. Immediately after, a post-creation schema modification (*ALTER TABLE*) was executed to apply the *CHECK (deal_type IN ('S', 'P'))* constraint directly onto this physical storage table.
- **(BR2) Sex Restriction:** Successfully enforced on the animals table via *CHECK (sex IN ('M', 'F'))*.
- **(BR3) Strictly Positive Values:** Successfully enforced across multiple tables for financial and physical attributes: *barn_area_chk*, *tool_price_chk*, *emp_salary_chk*, and *animal_weight_chk* all mandate values strictly greater than zero.

Triggers and Procedures

The remaining Business Rules entail complex logic. Consequently, these rules must be implemented programmatically using PL/SQL Triggers integrated with some procedures.

- **(BR4) Barn Capacity Limit:** Requires a trigger to read the barn's area, calculate the maximum threshold, and dynamically block an animal's insertion if the *current_occupancy* limit is breached.
- **(BR5) Chronological Order:** Requires triggers to validate dates against *SYSDATE* (for tool usage) and cross-reference dates between the animal's birth date and its transaction records.
- **(BR6 & BR7) Deal Limitations:** Require trigger logic to ensure an animal is involved in a specific transaction type a maximum of one time during its database lifecycle.
- **(BR8) Historical Immutability:** Requires checks within the trigger to guarantee that once a transaction is finalized, its core financial attributes (price and date) cannot be subsequently altered or secretly overwritten.
- **(BR9) Automated Space Vacation:** Requires a cascading active rule where a finalized Sale transaction autonomously triggers an update to vacate the animal from its barn, consequently firing BR4 to decrement the barn's *current_occupancy*.

Before developing and compiling the PL/SQL triggers, it is methodologically essential to populate the schema with a baseline set of test data. Implementing active rules on an empty database renders it impossible to accurately validate their behavior (e.g., verifying if a trigger correctly calculates the current occupancy requires a pre-existing barn with a defined area). Therefore, a preliminary data insertion phase will precede the PL/SQL implementation.

```
-- Large barn: area 100 -> max capacity 25 animals
INSERT INTO barns (barn_id, area, barn_location, current_occupancy)
VALUES ('B01', 100, 'North Wing', 0);

-- Small barn: area 12 -> max capacity 3 animals
INSERT INTO barns (barn_id, area, barn_location, current_occupancy)
VALUES ('B02', 12, 'South Wing', 3);

INSERT INTO tools (code, price, tool_type, tool_description, barn_ref)
VALUES (
    'T-100',
    50.00,
    'Pitchfork',
    'Standard pitchfork',
    (SELECT REF(b) FROM barns b WHERE b.barn_id = 'B01')
);
```

```

INSERT INTO employees (tax_code, emp_name, surname, salary, emp_address, tools, animals)
VALUES (
    'RSSMRA80A01H501U',
    'Mario',
    'Rossi',
    2500.00,
    t_address('Via Roma 1', 'Bari', 'BA', '70100'),
    t_usage_nt(), -- Constructor: initially empty Nested Table
    t_cured_nt() -- Constructor: initially empty Nested Table
);

INSERT INTO animals (code, weight, sex, birth, animal_type, last_shoeing, barn_ref)
VALUES (
    'HORSE-01',
    450.00,
    'M',
    TO_DATE('2020-05-15', 'YYYY-MM-DD'),
    'Horse',
    TO_DATE('2026-01-10', 'YYYY-MM-DD'),
    (SELECT REF(b) FROM barns b WHERE b.barn_id = 'B01')
);

INSERT INTO external_companies (vat, comp_name, comp_address, phones, deals)
VALUES (
    'IT12345678901',
    'AgriFarm S.p.A.',
    t_address('Via Milano 10', 'Milano', 'MI', '20100'),
    t_phone_varray('02-1234567', '333-9876543'),
    t_deal_nt() -- Constructor: initially empty Nested Table
);

UPDATE barns
SET current_occupancy = 1
WHERE barn_id = 'B01';

COMMIT;

```

Barn Capacity Limit (BR4)

To enforce the dynamic constraint regarding barn capacity, the trigger **manage_barn_capacity** was developed. This trigger monitors *INSERT*, *UPDATE* of *barn_ref*, and *DELETE* operations on the animals table.

The trigger was specifically bound to the *update of barn_ref* event, preventing unnecessary execution overhead during routine updates of unrelated attributes (such as an animal's weight). Furthermore, during this event, the trigger utilizes a boolean flag (*is_moving*) to compare the *:NEW.barn_ref* and *:OLD.barn_ref* identifiers. This guarantees that the complex occupancy logic is only executed if the animal is actually transferred to a different physical barn.

Before an animal is assigned to a new barn, the trigger dynamically calculates the maximum capacity of the target barn (*TRUNC(area / 4)*). If the current occupancy meets or exceeds this threshold, the transaction is forcefully aborted using a custom exception (**RAISE_APPLICATION_ERROR**).

To maintain data consistency without requiring manual updates, the trigger automatically increments the *current_occupancy* of the target barn and decrements the occupancy of the previous barn (in case of a *DELETE* or a valid *UPDATE* event flagged by *is_moving*).

The constraint was comprehensively validated through two distinct scenarios:

- *Successful Insertion*: A new animal (*SWINE-03*) was inserted into the large barn (*B01*, *max capacity*: 25). The trigger allowed the transaction and automatically incremented *B01*'s current occupancy to 2. This autonomous update was verified via a subsequent *SELECT* query.
- *Violation via Update*: An attempt was made to *UPDATE* the *barn_ref* of the newly inserted *SWINE-03* to transfer it into the small barn (*B02*). Since *B02* has a maximum capacity of 3 and was initialized with a *current_occupancy* of 3, it was strictly full. As expected, the trigger intercepted the *is_moving* state, evaluated the capacity breach, and blocked the transaction, raising the application error.

```
CREATE OR REPLACE TRIGGER manage_barn_capacity
BEFORE INSERT OR UPDATE OF barn_ref OR DELETE ON animals
FOR EACH ROW
DECLARE
    area NUMBER;
    occupancy INTEGER;
    max_capacity INTEGER;
    is_moving BOOLEAN := FALSE;
BEGIN
    -- Check if the animal is moving between different barns (UPDATE)
    IF UPDATING THEN
        IF (:NEW.barn_ref IS NULL AND :OLD.barn_ref IS NOT NULL) OR
           (:NEW.barn_ref IS NOT NULL AND :OLD.barn_ref IS NULL) OR
           (:NEW.barn_ref != :OLD.barn_ref) THEN
            is_moving := TRUE;
        END IF;
    END IF;

    -- Check capacity and INCREMENT new barn (INSERT or MOVING to a new barn)
    IF INSERTING OR (UPDATING AND is_moving) THEN
        IF :NEW.barn_ref IS NOT NULL THEN

            -- Retrieve area and current occupancy of the NEW barn
            SELECT b.area, b.current_occupancy
            INTO area, occupancy
            FROM barns b
            WHERE REF(b) = :NEW.barn_ref;

            -- Calculate max capacity (1 animal per 4 sq meters)
            max_capacity := TRUNC(area / 4);

            -- Block if the new barn is full
            IF occupancy >= max_capacity THEN
                RAISE_APPLICATION_ERROR(-20001, 'OPERATION BLOCKED:
                The target barn has reached its maximum capacity.');
```

```

-- DECREMENT old barn (DELETE or MOVING away from old barn)
IF DELETING OR (UPDATING AND is_moving) THEN
    IF :OLD.barn_ref IS NOT NULL THEN

        -- Automatically decrement the old barn's occupancy
        UPDATE barns b
        SET b.current_occupancy = b.current_occupancy - 1
        WHERE REF(b) = :OLD.barn_ref;

    END IF;
END IF;

END;
/

INSERT INTO animals (code, weight, sex, birth, animal_type, last_shoeing, barn_ref)
VALUES ('SWINE-03', 10.00, 'M', SYSDATE, 'Swine', NULL,
        (SELECT REF(b) FROM barns b WHERE b.barn_id = 'B01'));

-- Check the count increase
SELECT barn_id, current_occupancy FROM barns WHERE barn_id = 'B01';

UPDATE animals
SET barn_ref = (SELECT REF(b) FROM barns b WHERE b.barn_id = 'B02')
WHERE code = 'SWINE-03';

```

Chronological Order (BR5)

To enforce strict chronological consistency across the database, some triggers were programmatically implemented. The core challenge of this requirement stems from the database schema architecture: while the *last_shoeing* date resides in the main object table (*animals*), the tool usage dates (*date_use*) and transaction dates (*date_deal*) are encapsulated **within Nested Tables**.

In an Oracle database, defining a trigger directly on the physical storage table of a nested table is strictly prohibited by the database engine, resulting in an *ORA-22812: cannot reference nested table column storage table* exception. To overcome this systemic constraint, the active validation rules refer directly to the parent tables (*employees* and *external_companies*).

Three triggers were compiled:

1. **check_shoeing_date**: Bound to the *animals* table. It restricts the evaluation exclusively to entities where *animal_type* = 'Horse', ensuring that the *last_shoeing* attribute never contains a future date relative to the system timestamp (SYSDATE).
2. **check_tool_usage_date**: Bound to the parent *employees* table. It verifies that the collection is initialized and populated (:NEW.tools IS NOT NULL AND :NEW.tools.COUNT > 0). It then programmatically iterates through the collection elements using a *PL/SQL FOR loop* to block future usage dates. To optimize subsequent database debugging, the raised exception dynamically

appends the precise collection index (*Index: ' || i*) where the violation occurred.

3. **check_deal_date**: Bound to the parent *external_companies* table. For each transaction within the deals nested table, it asserts two conditions: the transaction date must not precede the animal's birth (*date_deal < birth*), and it cannot be logged in the future (*date_deal > SYSDATE*).

A further Oracle constraint (*ORA-25006*) prevents the use of the *UPDATE OF <column>* clause for Nested Table and Object Type columns within trigger definitions. As a result, the column-specific trigger optimization applicable to scalar columns (as used in *check_shoeing_date* with *UPDATE OF last_shoeing*) cannot be replicated for collection-based attributes. The triggers on employees and *external_companies* are therefore bound to the entire row, firing on any *INSERT OR UPDATE* operation regardless of which attribute is modified.

```
-- Check last_shoeing date for Horses
CREATE OR REPLACE TRIGGER check_shoeing_date
BEFORE INSERT OR UPDATE OF last_shoeing ON animals
FOR EACH ROW
BEGIN
    -- Check only if it is a Horse and the date is provided
    IF :NEW.animal_type = 'Horse' AND :NEW.last_shoeing IS NOT NULL THEN
        IF :NEW.last_shoeing > SYSDATE THEN
            RAISE_APPLICATION_ERROR(-20002,
                'INVALID DATE: The last shoeing date cannot be in the future.');
        END IF;
    END IF;
END;
/

-- Check tool usage date
CREATE OR REPLACE TRIGGER check_tool_usage_date
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
    IF :NEW.tools IS NOT NULL AND :NEW.tools.COUNT > 0 THEN
        FOR i IN 1 .. :NEW.tools.COUNT LOOP
            IF :NEW.tools(i).date_use > SYSDATE THEN
                RAISE_APPLICATION_ERROR(-20003,
                    'INVALID DATE: Tool usage date cannot be in the future.
                    Index: ' || i);
            END IF;
        END LOOP;
    END IF;
END;
/
```

```

-- Check deal date vs animal birth date
CREATE OR REPLACE TRIGGER check_deal_date
BEFORE INSERT OR UPDATE ON external_companies
FOR EACH ROW
DECLARE
    birth DATE;
BEGIN
    IF :NEW.deals IS NOT NULL AND :NEW.deals.COUNT > 0 THEN
        FOR i IN 1 .. :NEW.deals.COUNT LOOP
            IF :NEW.deals(i).animal_ref IS NOT NULL THEN

                SELECT a.birth INTO birth
                FROM animals a
                WHERE REF(a) = :NEW.deals(i).animal_ref;

                IF :NEW.deals(i).date_deal < birth THEN
                    RAISE_APPLICATION_ERROR(-20004,
                        'INVALID DATE: Deal date cannot be earlier than the animal birth date.');

```

The active rules were rigorously validated using a balanced testing matrix designed to evaluate both valid operations and boundary violations:

- **Test 1a & 1b (Horse Maintenance):** A past shoeing update succeeded seamlessly (1 row updated). Conversely, an attempt to schedule a shoeing in the year 2030 was immediately trapped, throwing an ORA-20002 custom application error.
- **Test 2a & 2b (Tool Tracking):** Updating the employee's tools collection with a timestamp of the current day was executed successfully. Attempting to inject a future record (tomorrow's date) caused the system to abort the transaction, explicitly raising an ORA-20003 error identifying the boundary violation at array Index 1.
- **Test 3a & 3b (Transaction Tracking):** Registering a purchase deal for SWINE-03 utilizing the current SYSDATE completed successfully, as it synchronized perfectly with the animal's birth record. Attempting to execute an update setting a transaction date in 2010 (*before the animal's date of birth*) was forcefully intercepted, returning an ORA-20004 integrity exception.

```

-- TEST 1a: EXPECTED: 1 row updated.
UPDATE animals
SET last_shoeing = TRUNC(SYSDATE) - 5
WHERE code = 'HORSE-01';

-- TEST 1b: Expected Error
UPDATE animals
SET last_shoeing = TO_DATE('2030-01-01', 'YYYY-MM-DD')
WHERE code = 'HORSE-01';

-- TEST 2a: EXPECTED: 1 row updated.
UPDATE employees
SET tools = t_usage_nt(
    t_usage((SELECT REF(t) FROM tools t WHERE t.code = 'T-100'), TRUNC(SYSDATE))
)
WHERE tax_code = 'RSSMRA80A01H501U';

-- TEST 2b: Expected Error
UPDATE employees
SET tools = t_usage_nt(
    t_usage((SELECT REF(t) FROM tools t WHERE t.code = 'T-100'), TRUNC(SYSDATE) + 1)
)
WHERE tax_code = 'RSSMRA80A01H501U';

-- TEST 3a: EXPECTED: 1 row updated.
UPDATE external_companies
SET deals = t_deal_nt(
    t_deal((SELECT REF(a) FROM animals a WHERE a.code = 'SWINE-03'), (SYSDATE), 150.00, 'P')
)
WHERE vat = 'IT12345678901';

-- TEST 3b: Expected Error
UPDATE external_companies
SET deals = t_deal_nt(
    t_deal((SELECT REF(a) FROM animals a WHERE a.code = 'SWINE-03'), TO_DATE('2010-01-01', 'YYYY-MM-DD'))
)
WHERE vat = 'IT12345678901';

-- Commit the valid transactions
COMMIT;

```

The parent-table trigger exposes a critical, native loophole within Oracle's Object-Relational abstraction layer. To prove this vulnerability, a **Limitation Test** was conducted. By utilizing direct collection manipulation syntax (*INSERT INTO TABLE(SELECT e.tools...)*), an invalid usage date set 10 days into the future was directly appended into the nested table.

Because this statement targets the storage table directly without modifying the parent row's structural reference, the parent trigger on employees was completely bypassed. The database engine accepted the corrupted data, returning a successful 1 row inserted confirmation message. (The transaction was immediately reverted using a *ROLLBACK* directive to restore the database to a consistent state).

```

-- Limitation Test: Expected: Trigger not activated -> Success
INSERT INTO TABLE(SELECT e.tools FROM employees e WHERE e.tax_code = 'RSSMRA80A01H501U')
VALUES (
    t_usage((SELECT REF(t) FROM tools t WHERE t.code = 'T-100'), TRUNC(SYSDATE) + 10)
);

-- Remove garbage data
ROLLBACK;

```

This empirical validation uncovers an essential security requirements conclusion: in the upcoming software architecture phase, data access and manipulation will be strictly abstracted. End-users and external interfaces will be forced to interface with the schema exclusively via PL/SQL Stored Procedures.

log_tool_usage Procedure

To systematically implement this strategy, I created the **log_tool_usage** Stored Procedure. Rather than inserting directly into the underlying nested storage table, the procedure encapsulates the modified array within a standard **UPDATE** statement targeting the parent *employees* row.

```

CREATE OR REPLACE PROCEDURE log_tool_usage (
    p_tax_code IN VARCHAR2,
    p_tool_code IN VARCHAR2,
    p_usage_date IN DATE DEFAULT SYSDATE
) AS
    tool_ref REF t_tool;
    usages t_usage_nt;
BEGIN
    SELECT REF(t) INTO tool_ref
    FROM tools t
    WHERE code = p_tool_code;

    SELECT tools INTO usages
    FROM employees
    WHERE tax_code = p_tax_code;

    -- Initialize the collection if it is currently empty
    IF usages IS NULL THEN
        usages := t_usage_nt();
    END IF;

    -- Extend the collection array and append the new tool usage record
    usages.EXTEND;
    usages(usages.LAST) := t_usage(tool_ref, TRUNC(p_usage_date));

    -- Execute insertion on the Parent Row.
    -- This forces the 'check_tool_usage_date' trigger to fire
    UPDATE employees
    SET tools = usages
    WHERE tax_code = p_tax_code;

    COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20022, 'DATA ERROR: Employee Tax Code or Tool Code not found.');
```

To validate the effectiveness of this procedure, a specific Test was executed. I inserted an employee (*TEST-EMP*) and a tool (*TEST-TOOL*). This ensured the test was self-contained and reproducible without relying on pre-existing transactional data.

log_tool_usage was invoked, attempting to intentionally replicate the parameters of the previous *Limitation Test* by logging a tool usage exactly 10 days into the future.

Unlike the direct manipulation approach, routing the request exclusively through the Stored Procedure successfully delegated the DML operation to the parent row. Consequently, the *check_tool_usage_date* trigger intercepted the violation, aborting the transaction and raising the expected application error.

```
-- TEST: Prove that the procedure forces the parent-trigger to fire.
INSERT INTO employees (tax_code, emp_name, surname, salary, emp_address, tools, animals)
VALUES (
  'TEST-EMP', 'Test', 'Procedure1', 2000.00,
  t_address('Via Test', 'Roma', 'RM', '00100'),
  t_usage_nt(), t_cured_nt()
);

INSERT INTO tools (code, price, tool_type, tool_description, barn_ref)
VALUES ('TEST-TOOL', 150.00, 'Test', 'Procedure1', NULL);

COMMIT;
BEGIN
  log_tool_usage('TEST-EMP', 'TEST-TOOL', TRUNC(SYSDATE) + 10);
END;
```

Deal Limitations and Space Vacation (BR6-7-8-9)

The final tier of the active database architecture manages the financial and biological lifecycle of the farm's livestock. To enforce Business Rules 6-7-8-9, an *AFTER INSERT OR UPDATE* row-level trigger **process_animal_deals** was implemented on the *external_companies* table.

The PL/SQL logic is structurally divided into three validation layers:

- **Intra-Row Validation (BR6 & BR7):** The trigger first iterates through the nested *deals* collection. By evaluating the collection matrix before it is committed, it actively blocks attempts to register identical transaction types (e.g., two purchases for the exact same animal).
- **Historical Immutability (BR8):** In an Object-Relational model, new elements are appended to a Nested Table via an UPDATE on the parent row. To prevent users from overwriting past transactions, the trigger compares scalar financial attributes (*price* and *date_deal*) between the *:NEW* and *:OLD* collections. It dynamically distinguishes between an *idempotent update* (e.g., a routine change in the company's address, which is safely bypassed) and an *historical overwrite* (which immediately raises an error).

- **Automated Space Vacation (BR9):** Once a transaction is validated as a new Sale, the trigger executes an update on the root *animals* table, explicitly setting the sold animal's **barn_ref** to **NULL**. Mutating this reference fires the *manage_barn_capacity* trigger (BR4), detecting the physical movement and autonomously decrementing the *current_occupancy* attribute of the corresponding *barns* object.

```

CREATE OR REPLACE TRIGGER process_animal_deals
AFTER INSERT OR UPDATE ON external_companies
FOR EACH ROW
DECLARE
    is_new BOOLEAN;
BEGIN
    -- If the collection contains transactions
    IF :NEW.deals IS NOT NULL AND :NEW.deals.COUNT > 0 THEN
        FOR i IN 1 .. :NEW.deals.COUNT LOOP
            IF :NEW.deals(i).animal_ref IS NOT NULL THEN
                -- Block duplicates (e.g., two purchases or two sales)
                FOR j IN 1 .. :NEW.deals.COUNT LOOP
                    IF j != i
                        AND :NEW.deals(j).deal_type = :NEW.deals(i).deal_type
                        AND :NEW.deals(j).animal_ref = :NEW.deals(i).animal_ref THEN
                        RAISE_APPLICATION_ERROR(-20010,
                            'Duplicate deal (' || :NEW.deals(i).deal_type || ') for
                             the same animal in this transaction.');
```

To verify the robustness of this active architecture, a small suite of tests was executed. The console outputs confirmed that the triggers correctly validate business constraints and manage cascading entity lifecycles.

1. Attempting to simultaneously insert two 'P' deals for the same animal (*SWINE-01*) resulted in a successful block.
2. Processing a valid 'S' (Sale) deal for *HORSE-01* successfully verified the automated space vacation. The queries confirmed that *HORSE-01*'s *barn_ref* was cleared to *NULL*, and the respective barn's occupancy decreased from 2 to 1 without any manual intervention.
3. Attempting to append a second 'S' deal for *HORSE-01* to bypass lifecycle limits was successfully intercepted. The system recognized the semantic duplication against the :OLD collection and aborted the transaction.
4. Registering an 'S' deal for *SWINE-03* (an animal that already possessed a valid 'P' deal in its history) was processed successfully. The trigger correctly differentiated between 'P' and 'S', recognizing the dual-deal scenario as a valid biological lifecycle completion. The new sale was accepted, subsequently triggering the barn vacation logic.

```

-- TEST 1
-- EXPECTED ERROR 20010
UPDATE external_companies
SET deals = t_deal_nt(
  t_deal((SELECT REF(a) FROM animals a WHERE a.code = 'SWINE-01'), TRUNC(SYSDATE), 100.00, 'P'),
  t_deal((SELECT REF(a) FROM animals a WHERE a.code = 'SWINE-01'), TRUNC(SYSDATE), 120.00, 'P')
)
WHERE vat = 'IT12345678901';

-- TEST 2
-- EXPECTED: 1 row updated, animal's barn_ref becomes NULL, barn's occupancy decreases.
-- State BEFORE sale
SELECT code, barn_ref FROM animals WHERE code = 'HORSE-01';
SELECT barn_id, current_occupancy FROM barns WHERE barn_id = 'B01';

-- Execute sale
UPDATE external_companies
SET deals = t_deal_nt(
  t_deal((SELECT REF(a) FROM animals a WHERE a.code = 'HORSE-01'), TRUNC(SYSDATE), 2500.00, 'S')
)
WHERE vat = 'IT12345678901';

-- State AFTER sale (Verify BR4 and BR7 fired)
SELECT code, barn_ref FROM animals WHERE code = 'HORSE-01';
SELECT barn_id, current_occupancy FROM barns WHERE barn_id = 'B01';

COMMIT;

-- TEST 3
-- EXPECTED ERROR 20012
UPDATE external_companies
SET deals = t_deal_nt(
  t_deal((SELECT REF(a) FROM animals a WHERE a.code = 'HORSE-01'), TRUNC(SYSDATE), 2500.00, 'S'),
  t_deal((SELECT REF(a) FROM animals a WHERE a.code = 'HORSE-01'), TRUNC(SYSDATE), 3000.00, 'S')
)
WHERE vat = 'IT12345678901';

UPDATE animals SET birth = TO_DATE('2020-01-01', 'YYYY-MM-DD') WHERE code = 'SWINE-03';
COMMIT;

-- TEST 4: Valid Lifecycle (Purchase and Sale of the same animal)
-- EXPECTED: 1 row updated (Success). The trigger correctly allows different deal types ('P' and 'S').
-- The 'S' deal will also trigger BR7, automatically vacating SWINE-03 from its barn.
-- State BEFORE lifecycle completion
SELECT code, barn_ref FROM animals WHERE code = 'SWINE-03';

-- Execute valid dual-deal (Preserving the old 'P' and adding a new 'S')
UPDATE external_companies
SET deals = t_deal_nt(
  t_deal((SELECT REF(a) FROM animals a WHERE a.code = 'SWINE-03'), TRUNC(SYSDATE), 150.00, 'P'),
  t_deal((SELECT REF(a) FROM animals a WHERE a.code = 'SWINE-03'), TRUNC(SYSDATE), 200.00, 'S')
)
WHERE vat = 'IT12345678901';

-- State AFTER lifecycle completion
-- SWINE-03's barn_ref MUST be NULL now
SELECT code, barn_ref FROM animals WHERE code = 'SWINE-03';

COMMIT;

```

While the trigger reliably prevents duplicate operations for the same physical entity reference and permits a *'Purchase-to-Sale'* lifecycle, it cannot structurally enforce the chronological depreciation of an entity across different companies (e.g., preventing a new 'Purchase' on a reference that was already globally subjected to a 'Sale').

Enforcing the business rule that a repurchased animal must be initialized as a strictly new distinct entity is delegated to a stored procedure. It also resolves the cross-company validation that was architecturally impossible at the trigger level due to Oracle's mutating table restriction.

register_animal_deal Procedure

To systematically resolve this limitation, the **register_animal_deal** Stored Procedure was implemented. Before encapsulating the new record and routing it to the parent row (*which guarantees the trigger fires*), the procedure performs a lookup across the nested tables of registered companies. If an attempt is made to register a Purchase ('P') for an animal REF that has already been marked as Sold ('S') globally, the procedure aborts the transaction, enforcing the absolute requirement that repurchased animals must be instantiated as entirely new database entities.

```
CREATE OR REPLACE PROCEDURE register_animal_deal (
    p_vat IN VARCHAR2,
    p_animal_code IN VARCHAR2,
    p_deal_type IN CHAR,
    p_price IN NUMBER,
    p_date IN DATE DEFAULT SYSDATE
) AS
    animal_ref REF t_animal;
    deals t_deal_nt;
    sold_count NUMBER;
BEGIN
    SELECT REF(a) INTO animal_ref
    FROM animals a
    WHERE code = p_animal_code;

    -- If registering a Purchase ('P'), ensure the entity wasn't
    -- previously Sold ('S') globally
    IF p_deal_type = 'P' THEN
        SELECT COUNT(*) INTO sold_count
        FROM external_companies ec, TABLE(ec.deals) d
        WHERE d.animal_ref = animal_ref AND d.deal_type = 'S';

        IF sold_count > 0 THEN
            RAISE_APPLICATION_ERROR(-20020,
                'DOMAIN INTEGRITY VIOLATION: This animal was previously sold.
                To repurchase, you must register a new animal entity.');
        END IF;
    END IF;

    -- Retrieve the current collection, extend it, and append the new deal
    SELECT deals INTO deals
    FROM external_companies
    WHERE vat = p_vat;
```

```

-- Initialize the collection if it is currently empty
IF deals IS NULL THEN
    deals := t_deal_nt();
END IF;

deals.EXTEND;
deals(deals.LAST) := t_deal(animal_ref, TRUNC(p_date), p_price, p_deal_type);

-- Execute Update on the Parent Row to guarantee Trigger execution
UPDATE external_companies
SET deals = deals
WHERE vat = p_vat;

COMMIT;
EXCEPTION
WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20021, 'External Company VAT or Animal Code
    not found.');
```

To validate the domain integrity enforcement, a test was executed. The test simulates a valid initial 'Sale' of an animal, followed by an attempt to physically 'Purchase' the same animal.

```

-- TEST REPURCHASE
INSERT INTO animals (code, weight, sex, birth, animal_type, last_shoeing, barn_ref)
VALUES ('TEST-HORSE', 450.00, 'M', TO_DATE('2020-01-01', 'YYYY-MM-DD'), 'Horse', NULL, NULL);

INSERT INTO external_companies (vat, comp_name, comp_address, phones, deals)
VALUES ('TEST-VAT', 'Global Trading Corp', t_address('Main St', 'Milan', 'MI', '20100'), t_phone_varray(), t_deal_nt());

COMMIT;

-- Register a standard Sale ('S').
BEGIN
    register_animal_deal('TEST-VAT', 'TEST-HORSE', 'S', 2500.00, TRUNC(SYSDATE));
END;
/

-- Attempt to register a Purchase ('P') for the same animal.
BEGIN
    register_animal_deal('TEST-VAT', 'TEST-HORSE', 'P', 1500.00, TRUNC(SYSDATE) + 1);
END;
/
```

The procedure successfully intercepted the violation and the transaction was aborted.

Views

To satisfy two of the most frequent queries (*Operations 2 and 4*), a presentation layer consisting of relational Views was engineered.

Barn Tools Inventory (Operation 2)

According to the workload estimations, the farm management requires a daily inventory report detailing the tools located in each specific barn (estimated at 30 executions per day across 60 barns and 600 tools). The **v_barn_tools** view was created to satisfy this requirement. By leveraging the *DEREF* function on the *barn_ref* attribute within the tools table, the view projects a flat table integrating physical tool attributes with their associated physical barn identifiers.

```
CREATE OR REPLACE VIEW v_barn_tools AS
SELECT Deref(t.barn_ref).barn_id AS barn_id,
       Deref(t.barn_ref).barn_location AS location,
       Deref(t.barn_ref).area AS barn_area,
       t.code AS tool_code,
       t.tool_type,
       t.tool_description,
       t.price
FROM tools t
WHERE t.barn_ref IS NOT NULL
ORDER BY barn_id, tool_type;
```

Example of usage:

```
SELECT * FROM v_barn_tools;
```

BARN_ID	LOCATION	BARN_AREA	TOOL_CODE
B01	North Wing	100	T-100
TOOL_TYPE	TOOL_DESCRIPTION	PRICE	
Pitchfork	Standard pitchfork	50	

Employee Tool Usage (Operation 4)

Operation 4 demands a comprehensive report of all tools utilized by maintenance personnel, including the dates of usage and the physical location of the respective tools. The view **v_employee_tool** was developed.

This view unnests the *tools* collection from the parent *employees* row, dereferences the *tool_ref* to access the tool's characteristics, and performs a secondary cascaded dereference (*DEREF(Deref(u.tool_ref).barn_ref)*) to retrieve the specific barn where the tool is currently housed.

```

CREATE OR REPLACE VIEW v_employee_tool AS
SELECT e.tax_code AS employee_tax_code,
       e.surname,
       e.emp_name,
       u.date_use,
       Deref(u.tool_ref).code AS tool_code,
       Deref(u.tool_ref).tool_description AS description,
       Deref(u.tool_ref).tool_type AS type,
       Deref(Deref(u.tool_ref).barn_ref).barn_id AS located_in_barn,
       Deref(Deref(u.tool_ref).barn_ref).barn_location AS barn_location
FROM employees e, TABLE(e.tools) u

```

Example of usage:

```
SELECT * FROM v_employee_tool;
```

EMPLOYEE_TAX_CODE	SURNAME	EMP_NAME	DATE_USE	TOOL_CODE
RSSMRA80A01H501U	Rossi	Mario	22-GIU-26	T-100

DESCRIPTION	TYPE	LOCATED_IN_BARN	BARN_LOCATION
Standard pitchfork	Pitchfork	B01	North Wing

Physical Design

The physical optimization of the system was driven by an analysis of the most frequent workloads (specifically *Operations 2 and 4*). To allow the Cost-Based Optimizer to generate realistic execution plans, the database was populated with the exact initial volumes estimated for the farm's start-up phase (*60 barns, 600 tools, and 20 maintenance employees*).

Operation 2 requires, for each barn, the list of tools available including their description, type, and purchase price. It is executed 30 times per day and is supported by the view *v_barn_tools*, which joins the *tools* table with the *barns* table through *DEREF* on the *barn_ref* REF column.

Operation 4 requires, for each maintenance employee, the complete list of all tools they have used, together with each tool's description, type, the barn where it is stored, and the date of each use. It is also executed 30 times per day and is supported by the view *v_employee_tool*, which navigates into each employee's nested table of usages, then performs a double *DEREF* chain, first to resolve the tool, then to resolve the barn in which that tool is located.

In order to produce execution plans that reflect realistic optimizer behavior, the database was populated with row counts consistent with the Volumes Table. The insertion was carried out through a series of PL/SQL blocks, executed in an order dictated by referential integrity: since tools hold REF columns pointing to barns, the

barn rows had to exist before any tool could be inserted. Similarly, tool usage records in the nested table required both the parent employees and the referenced tools to already exist in the database. 60 barns were inserted first, followed by 600 tools distributed evenly across them (10 per barn), then 20 maintenance employees initialized with empty nested tables, and finally 200 tool usage records per employee inserted directly into the nested table using the *INSERT INTO TABLE(SELECT ...)* syntax. The total of 4,000 usage records in *emp_usages_nt_tab* is smaller than the declared 54,000, but was deemed sufficient for the CBO to produce representative cost estimates.

After all insertions, table statistics were refreshed. Without up-to-date statistics, the CBO operates on stale or default estimates and may generate plans that do not reflect the actual data distribution.

```
-- Insert 60 Barns
BEGIN
  FOR i IN 1..60 LOOP
    INSERT INTO barns VALUES (
      t_barn('BARN' || i, 500, 'Farm Sector ' || i, 0)
    );
  END LOOP;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Inserted 60 barns.');
```

```
-- Insert 600 Tools (~10 tools distributed per barn)
BEGIN
  FOR i IN 1..600 LOOP
    INSERT INTO tools VALUES (
      t_tool(
        'TOOL' || i,
        50.00,
        'Generic Tool',
        'Standard Equipment',
        (SELECT REF(b) FROM barns b WHERE b.barn_id = 'BARN' || TO_CHAR(MOD(i,60)+1))
      )
    );
  END LOOP;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Inserted 600 tools.');
```

```
END;
/
```

```

-- Insert 20 Maintenance Employees
BEGIN
  FOR i IN 1..20 LOOP
    INSERT INTO employees VALUES (
      t_employee(
        'EMP'||i,
        'John',
        'Doe '||i,
        2000.00,
        t_address('Main St', 'City', 'PR', '00000'),
        t_usage_nt(),
        t_cured_nt()
      )
    );
  END LOOP;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Inserted 20 employees.');
```

```

END;
/

-- Insert 200 Tool Usages per Employee (4000 total records in Nested Table)
BEGIN
  FOR i IN 1..20 LOOP
    FOR j IN 1..200 LOOP
      INSERT INTO TABLE(
        SELECT e.tools FROM employees e WHERE e.tax_code = 'EMP'||i
      ) VALUES (
        t_usage(
          (SELECT REF(t) FROM tools t WHERE t.code = 'TOOL'||TO_CHAR(MOD(j,600)+1)).
          SYSDATE - 1
        )
      );
    END LOOP;
    -- Commit per employee
    COMMIT;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Inserted 4000 tool usages.');
```

```

END;
/

-- UPDATE STATISTICS: Required to allow the Cost-Based Optimizer to recognize the new data v
-----
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'BARNs');
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'TOOLS');
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'ANIMALS');
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'EMPLOYEES');
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'EMP_USAGES_NT_TAB');
```

Before introducing any custom index, execution plans were collected for the two target operations using *EXPLAIN PLAN FOR* followed by *DBMS_XPLAN.DISPLAY*. These plans establish the baseline against which the effect of the physical design can be measured.

Operation 2: Tool in a specific Barn

The query underlying Operation 2 filters the *tools* table by *barn_ref*, matching it against the REF of a specific barn:

```
SELECT 'OP2 AFTER INDEX: Tools in BARN5' AS status FROM DUAL;
EXPLAIN PLAN FOR
  SELECT t.code, t.tool_type, t.tool_description, t.price
  FROM tools t
  WHERE t.barn_ref = (SELECT REF(b) FROM barns b WHERE b.barn_id = 'BARN5');
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

The optimizer produced a plan involving a **full table scan** of the tools table, at a **cost of 4**, estimating a result set of 10 rows.

PLAN_TABLE_OUTPUT

Plan hash value: 3745379016

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		10	610	5 (0)	00:00:01
* 1	TABLE ACCESS FULL	TOOLS	10	610	4 (0)	00:00:01
2	TABLE ACCESS BY INDEX ROWID	BARN5	1	36	1 (0)	00:00:01
* 3	INDEX UNIQUE SCAN	BARN_PK	1		0 (0)	00:00:01

Operation 4: Tool usage for a specific employee

The query underlying Operation 4 reads through the nested table of a given employee:

```
-- OP4: Complete tool history for a specific employee
SELECT 'OP4 BEFORE INDEX: Tool usages by EMP1' AS status FROM DUAL;
EXPLAIN PLAN FOR
  SELECT *
  FROM v_employee_tool
  WHERE employee_tax_code = 'EMP1';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

The plan produced for this query was, even at the baseline stage, already structured as an **index-driven nested loop**. Oracle first resolved the employee using an *INDEX UNIQUE SCAN* on the primary key *EMPLOYEE_PK* at **zero cost**, then joined the result with *emp_usages_nt_tab*. The total estimated cost was 5. This plan represents optimal behavior for this query: rather than scanning all 4,000 rows in the nested table storage, Oracle uses the *NESTED_TABLE_ID* index to directly retrieve only the 200 rows belonging to *EMP1*. This is an automatic index created by Oracle itself when the nested table storage table was defined.

PLAN_TABLE_OUTPUT

Plan hash value: 1140670995

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		191	11842	5 (20)	00:00:01
1	SORT ORDER BY		191	11842	5 (20)	00:00:01
2	NESTED LOOPS		191	11842	4 (0)	00:00:01
3	TABLE ACCESS BY INDEX ROWID	EMPLOYEES	1	37	1 (0)	00:00:01
* 4	INDEX UNIQUE SCAN	EMPLOYEE_PK	1		0 (0)	00:00:01
5	TABLE ACCESS BY INDEX ROWID BATCHED	EMP_USAGES_NT_TAB	191	4775	3 (0)	00:00:01

PLAN_TABLE_OUTPUT

* 6	INDEX RANGE SCAN	SYS_FK0000074737N00012\$	191		1 (0)	00:00:01
-----	------------------	--------------------------	-----	--	-------	----------

Index Creation

Based on the baseline analysis and on the declared production volumes, 2 indexes were designed.

```
-- Index Creation
```

```
CREATE INDEX idx_tools_barn_ref ON tools(barn_ref);
CREATE INDEX idx_emp_usages_tool_ref ON emp_usages_nt_tab(tool_ref);
```

1. **idx_tools_barn_ref** targets the access pattern of Operation 2. It is worth addressing an apparent ambiguity in the formulation of Operation 2: the description says *"for each barn, print the list of tools"*, which might suggest that the query always retrieves all barns at once and therefore a full scan is unavoidable. However, in a real operational context, this operation can also be executed **one barn at a time**. For instance, a barn manager can query the inventory of their specific barn, or a scheduled reporting job iterating through barns sequentially, one query per barn. In both cases, the query involves a selective filter on a single *barn_ref* value, and an index scan over 10 rows out of 600 is more efficient than a full scan.
2. Attempted index **idx_emp_usages_tool_ref**: Oracle rejected this statement with error *"ORA-02327: cannot create index on expression with data type REF"*. Oracle's B-Tree indexing engine does not support REF-typed columns as index keys, regardless of their cardinality or the use case. This is an architectural constraint of the Oracle object-relational model: REF values are internally represented as complex OID-based structures that cannot be directly used as B-Tree entries. The locking concern and the reverse-lookup use case therefore remain valid in theory but have no indexing remedy available in Oracle's current implementation.

Post-Index Execution Plans

After creating the two successful indexes and refreshing statistics, the same queries were re-submitted to *EXPLAIN PLAN* to assess the impact of the physical design.

Operation 2

```
-- EXPLAIN PLAN AFTER INDEXES
-- OP2 AFTER
SELECT 'OP2 AFTER INDEX: Tools in BARN5' AS status FROM DUAL;
EXPLAIN PLAN FOR
  SELECT t.code, t.tool_type, t.tool_description, t.price
  FROM tools t
  WHERE t.barn_ref = (SELECT REF(b) FROM barns b WHERE b.barn_id = 'BARN5');
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

The execution plan for Operation 2 remained unchanged after the index was created. The optimizer still chose a **full table scan of tools at cost 4**.

PLAN_TABLE_OUTPUT

Plan hash value: 3745379016

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		10	610	5 (0)	00:00:01
* 1	TABLE ACCESS FULL	TOOLS	10	610	4 (0)	00:00:01
2	TABLE ACCESS BY INDEX ROWID	BARN5	1	36	1 (0)	00:00:01
* 3	INDEX UNIQUE SCAN	BARN_PK	1		0 (0)	00:00:01

This outcome is not an indication that the index is incorrect but it is a cost-based decision made by the CBO given the current data volume. With only 600 rows distributed across a small number of data blocks, Oracle can retrieve all tool data in a single multi-block I/O pass. An index range scan, by contrast, would require traversing the B-Tree structure and then performing random single-block I/O for each matching row: an overhead that, at this scale, can exceed the cost of the full scan. At production scale, where the tools table may grow to tens of thousands of rows, the selectivity of a barn-specific filter would make the index access path significantly cheaper.

Operation 4

```
-- OP4 AFTER
SELECT 'OP4 AFTER INDEX: Tool usages by EMP1' AS status FROM DUAL;
EXPLAIN PLAN FOR
  SELECT *
  FROM v_employee_tool
  WHERE employee_tax_code = 'EMP1';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

The execution plan for Operation 4 also remained unchanged, for a different reason: it was already optimal before any index was created. The system-generated index on *NESTED_TABLE_ID* was already doing its job efficiently.

PLAN_TABLE_OUTPUT

Plan hash value: 1140670995

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		191	11842	5 (20)	00:00:01
1	SORT ORDER BY		191	11842	5 (20)	00:00:01
2	NESTED LOOPS		191	11842	4 (0)	00:00:01
3	TABLE ACCESS BY INDEX ROWID	EMPLOYEES	1	37	1 (0)	00:00:01
* 4	INDEX UNIQUE SCAN	EMPLOYEE_PK	1		0 (0)	00:00:01
5	TABLE ACCESS BY INDEX ROWID BATCHED	EMP_USAGES_NT_TAB	191	4775	3 (0)	00:00:01

PLAN_TABLE_OUTPUT

* 6	INDEX RANGE SCAN	SYS_FK00000074737N00012\$	191		1 (0)	00:00:01
-----	------------------	---------------------------	-----	--	-------	----------

Physical Storage Analysis

To provide a quantitative basis for interpreting the execution plan costs observed in the previous sections, it is useful to examine the actual physical configuration of the database and the storage layout of the tables involved. This analysis makes it possible to verify, with real numbers, why the Cost-Based Optimizer behaved as it did, and to estimate at what data volume the index access path would become preferable to a full table scan.

- The first parameter to examine is **the size of an Oracle data block**.
- The second parameter governs how many blocks Oracle reads in a single sequential I/O operation during a full table scan, known as a **multiblock read**.

```
-- Block Dimension
SHOW PARAMETER db_block_size;

-- Number of blocks for each I/O full scan
SHOW PARAMETER db_file_multiblock_read_count;
```

NAME	TYPE	VALUE
db_block_size	integer	8192
NAME	TYPE	VALUE
db_file_multiblock_read_count	integer	128

The block size is 8,192 bytes (**8 KB**). Multi-block read is set to **128 blocks per I/O call**, which means that during a full table scan Oracle reads up to $128 \times 8 \text{ KB} = 1 \text{ MB}$ in a single operation. This is a particularly high value and has a decisive effect on the optimizer's cost model: it makes full table scans extremely cheap, because even tables of several hundred blocks can be read in just a handful of I/O calls.

Having established the I/O parameters, the physical statistics of the relevant tables were queried after statistics collection.

```
-- Tables statics
SELECT table_name, num_rows, blocks, avg_row_len
FROM user_all_tables
WHERE table_name IN ('TOOLS', 'EMPLOYEES', 'EMP_USAGES_NT_TAB', 'ANIMALS', 'BARNS');
```

TABLE_NAME	NUM_ROWS	BLOCKS	AVG_ROW_LEN
ANIMALS	1503	15	60
BARNS	62	1	45
EMPLOYEES	22	1	94
EMP_USAGES_NT_TAB	4001	39	25
TOOLS	602	7	77

The BARNS table occupies a single data block containing 62 rows. The EMPLOYEES table fits in a single block. The TOOLS table, which is the central target of Operation 2, holds 602 rows distributed across 7 data blocks, for a total footprint of approximately 56 KB (well under the 1 MB threshold of a single multiblock read). The EMP_USAGES_NT_TAB nested table storage, which is the structure accessed by Operation 4, holds 4,001 usage records across 39 blocks, still comfortably within a single multiblock I/O call.

Data Warehouse

To support the strategic decision-making process of Ranch Holdings, an analytical Data Warehouse environment was designed. While the transactional database manages the entirety of the farm's daily operations, including barn maintenance and personnel logistics, the analytical scope of this Data Warehouse has been deliberately targeted toward the **commercial and financial workflows**. Specifically, the model tracks and evaluates the monetary movements generated by the purchase and sale of livestock.

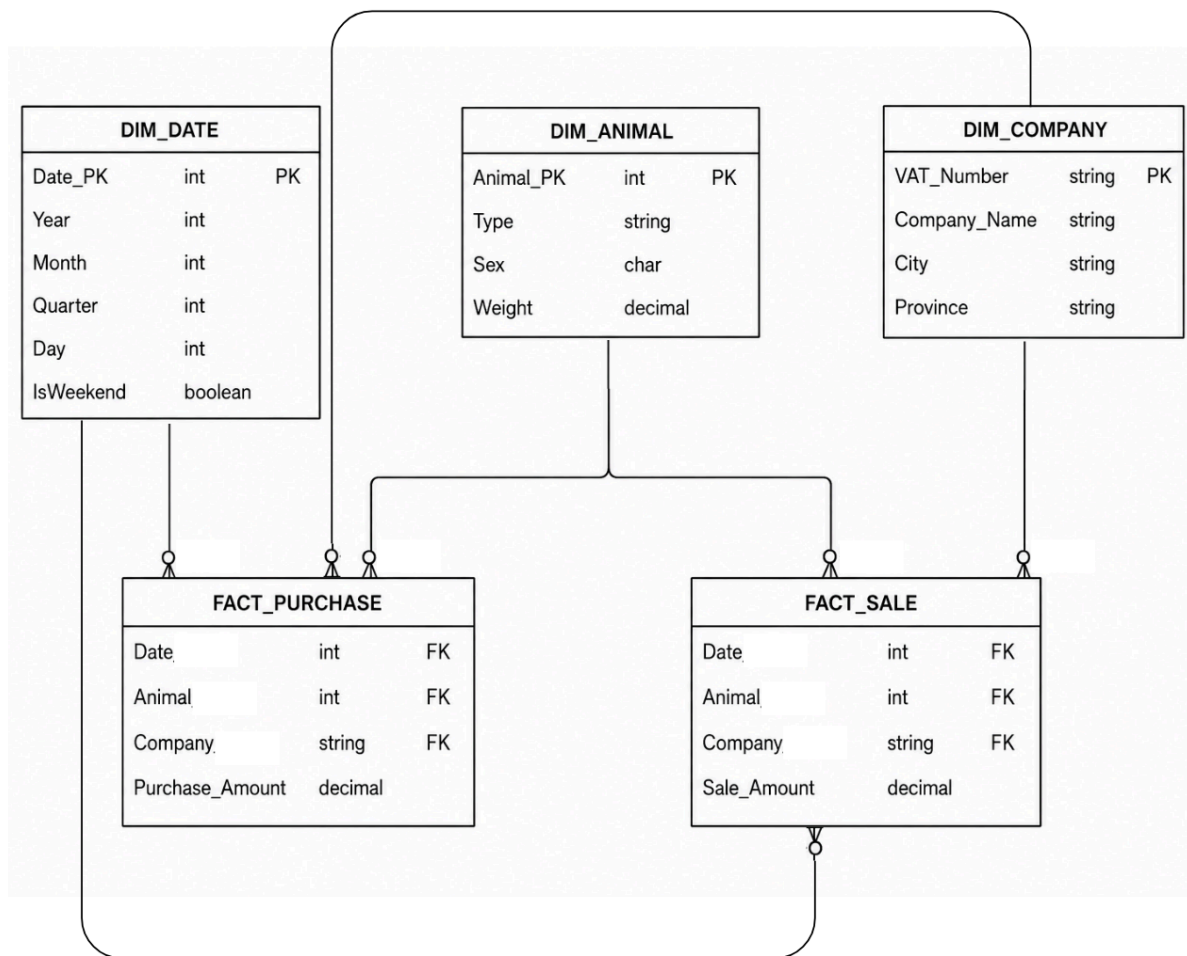
Constellation Diagram

Given the distinct nature of purchasing and selling events, the dimensional model was engineered using a **Constellation Diagram**. The core of the Constellation Schema consists of the two fact tables, both designed at the most detailed level of granularity (a single animal traded in a single day with a specific external company).

- **FACT_PURCHASE:**
 - *Foreign Keys:* Date, Animal, Company.
 - *Measure:* Purchase_Amount.
- **FACT_SALE:**
 - *Foreign Keys:* Date, Animal, Company.
 - *Measure:* Sale_Amount.

To allow consistent multidimensional analysis across the two different fact tables, three **Dimensions** were designed.

1. **DIM_DATE:** The temporal dimension allowing time-series analysis.
 - *Attributes:* Date_PK, Year, Month, Quarter, Day, IsWeekend.
- **DIM_ANIMAL:** The subject of the commercial transactions.
 - *Attributes:* Animal_PK, Type (Swine, Cattle, Horse), Sex, Weight.
- **DIM_COMPANY:** The external business partners involved in the deals.
 - *Attributes:* VAT_Number, Company_Name, City, Province.



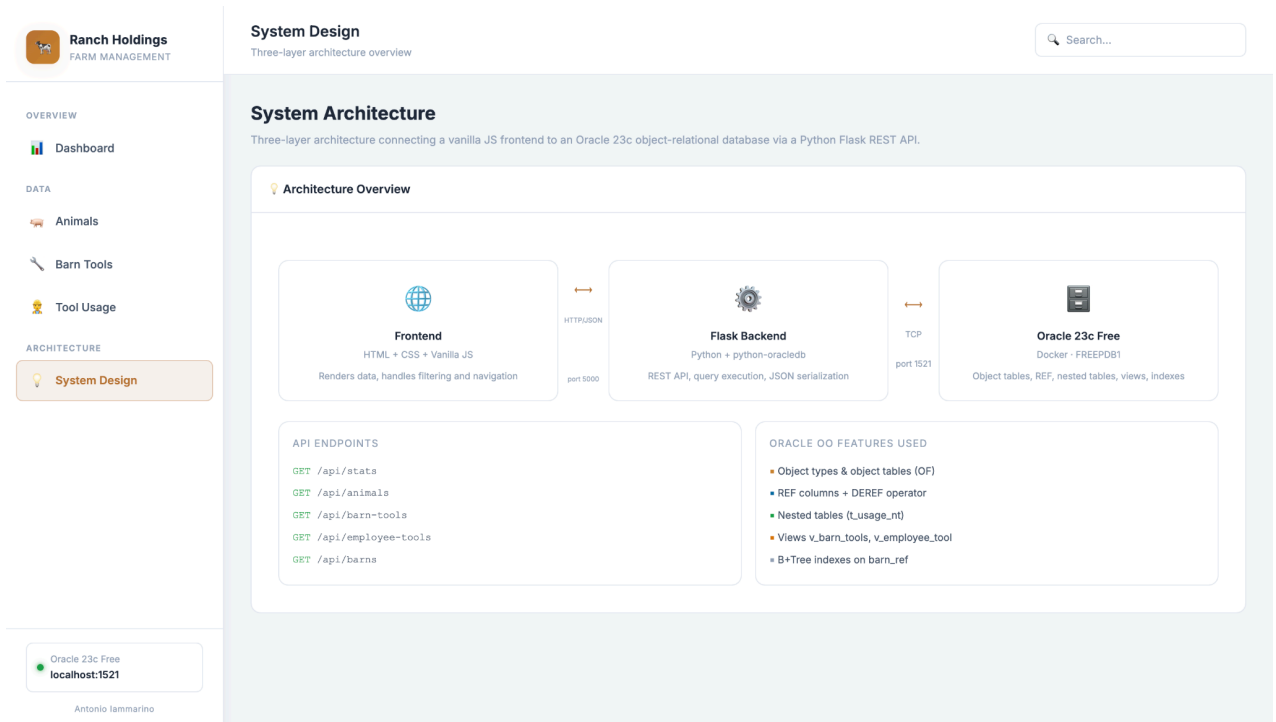
Web Application

To conclude the database project, a comprehensive web application was developed to demonstrate and visualize the implemented database features. The goal of this application is to showcase how the object-relational schema designed for the "Ranch Holdings" farm operates in a practical, real-world scenario. Through this dashboard, users can interact with the underlying database and monitor the farm's operations.

Architecture

The application is built using a modern *Three-Tier Architecture*, ensuring a clean separation of concerns:

1. **Frontend:** Developed using pure HTML5, CSS3, and JavaScript. It communicates with the backend asynchronously via the Fetch API using standard HTTP/JSON protocols.
2. **Backend:** Powered by Python with Flask. Exposes an API that processes frontend requests and formats the responses. It connects to the database using the official *oracledb* library.
3. **Database:** Hosted on *Oracle 23c Free* running within a Docker container.



Application Sections

The web application interface is divided into five main sections, accessible via a sidebar navigation menu:

Dashboard

Provides a high-level statistical overview of the farm's current state. It executes *COUNT* queries to display the total number of registered animals (broken down by type: Horse, Cattle, Swine), active barns, tools in inventory, employees, and external companies. It also includes a summary of the Object-Relational schema elements currently in use.



Ranch Holdings

FARM MANAGEMENT

OVERVIEW

Dashboard

DATA

Animals

Barn Tools

Tool Usage

ARCHITECTURE

System Design

Oracle 23c Free

localhost:1521

Antonio Iammarino

Dashboard

Farm overview and statistics



1503

TOTAL ANIMALS

2 Horse

1500 Cattle

1 Swine



62

BARNs



602

TOOLS



22

EMPLOYEES



2

EXTERNAL COMPANIES

Object-Relational Schema

Oracle 23c Free

OBJECT TABLES

animals OF t_animal

barns OF t_barn

tools OF t_tool

employees OF t_employee

REF COLUMNS

animal.barn_ref

tool.barn_ref

usage.tool_ref

VIEWS USED

v_barn_tools

v_employee_tool

INDEXES

idx_tools_barn_ref

Animals

Displays the complete registry of the farm's livestock and allows dynamic filtering by animal type and sex, as well as a global search by code or location.



Ranch Holdings

FARM MANAGEMENT

OVERVIEW

Dashboard

DATA

Animals

Barn Tools

Tool Usage

ARCHITECTURE

System Design

Oracle 23c Free

localhost:1521

Antonio Iammarino

Animals

All animals with barn assignment via Deref

Search...

Animals

All animals registered on the farm, with barn assignment via Deref on barn_ref.

SELECT a.*, Deref(a.barn_ref).* FROM animals a

Animal Registry

1503 records

All types

All sexes

CODE	TYPE	SEX	WEIGHT (KG)	BIRTH	LAST SHOEING	BARN	LOCATION
A1	Cattle	F	100.0	2025-01-01	N/A	BARN2	Farm Sector 2
A10	Cattle	F	100.0	2025-01-01	N/A	BARN11	Farm Sector 11
A100	Cattle	F	100.0	2025-01-01	N/A	BARN41	Farm Sector 41
A1000	Cattle	F	100.0	2025-01-01	N/A	BARN41	Farm Sector 41
A1001	Cattle	F	100.0	2025-01-01	N/A	BARN42	Farm Sector 42
A1002	Cattle	F	100.0	2025-01-01	N/A	BARN43	Farm Sector 43
A1003	Cattle	F	100.0	2025-01-01	N/A	BARN44	Farm Sector 44
A1004	Cattle	F	100.0	2025-01-01	N/A	BARN45	Farm Sector 45
A1005	Cattle	F	100.0	2025-01-01	N/A	BARN46	Farm Sector 46
A1006	Cattle	F	100.0	2025-01-01	N/A	BARN47	Farm Sector 47

Barn Tools

Monitors the inventory of maintenance tools available across the various barns. Users can filter the inventory by a specific Barn ID or Tool Type to quickly locate equipment.

 **Ranch Holdings**
FARM MANAGEMENT

OVERVIEW

Dashboard

DATA

Animals

Barn Tools

Tool Usage

ARCHITECTURE

System Design

Barn Tools

Tool inventory per barn — v_barn_tools view

Search...

Barn Tools

Tools available in each barn. Data sourced from the v_barn_tools view using DEREf on barn_ref.

SELECT FROM v_barn_tools

Tool Inventory by Barn

1 records

B01 All tool types

BARN ID	LOCATION	AREA (M²)	TOOL CODE	TYPE	DESCRIPTION	PRICE (€)
B01	North Wing	100 m²	T-100	Pitchfork	Standard pitchfork	€ 50.00

Tool Usage

Tracks the historical usage of tools by the farm's employees. Allows to filter tool usage logs by specific employees.

 **Ranch Holdings**
FARM MANAGEMENT

OVERVIEW

Dashboard

DATA

Animals

Barn Tools

Tool Usage

Architecture

System Design

Tool Usage

Employee tool usage history — v_employee_tool view

Search...

Tool Usage History

Tool usage records per employee via the v_employee_tool view — double DEREf chain: usage → tool → barn.

SELECT FROM v_employee_tool

Employee Tool Usage

200 records

Doe 7 John

EMPLOYEE	TAX CODE	DATE USED	TOOL CODE	DESCRIPTION	TYPE	BARN	LOCATION
Doe 7 John	EMP7	2026-06-22	TOOL26	Standard Equipment	Generic Tool	BARN27	Farm Sector 27
Doe 7 John	EMP7	2026-06-22	TOOL27	Standard Equipment	Generic Tool	BARN28	Farm Sector 28
Doe 7 John	EMP7	2026-06-22	TOOL28	Standard Equipment	Generic Tool	BARN29	Farm Sector 29
Doe 7 John	EMP7	2026-06-22	TOOL29	Standard Equipment	Generic Tool	BARN30	Farm Sector 30
Doe 7 John	EMP7	2026-06-22	TOOL30	Standard Equipment	Generic Tool	BARN31	Farm Sector 31
Doe 7 John	EMP7	2026-06-22	TOOL31	Standard Equipment	Generic Tool	BARN32	Farm Sector 32
Doe 7 John	EMP7	2026-06-22	TOOL32	Standard Equipment	Generic Tool	BARN33	Farm Sector 33
Doe 7 John	EMP7	2026-06-22	TOOL33	Standard Equipment	Generic Tool	BARN34	Farm Sector 34
Doe 7 John	EMP7	2026-06-22	TOOL58	Standard Equipment	Generic Tool	BARN59	Farm Sector 59
Doe 7 John	EMP7	2026-06-22	TOOL59	Standard Equipment	Generic Tool	BARN60	Farm Sector 60

Oracle 23c Free

localhost:1521

Antonio Iammarino